Gabarito — Primeiro Modelo ML

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

Carregamento do dataset "Iris"

- Finalidade: O objetivo do dataset é classificar flores de íris em uma das três espécies, com base em medidas de suas pétalas e sépalas.
- Tamanho: O conjunto contém 150 amostras (linhas) no total.
- Estrutura: Possui 5 colunas: 4 características numéricas (features) e 1 alvo categórico (target/espécie).

```
from sklearn.datasets import load_iris
iris = load_iris()
iris
```

```
the data.\n  - Many, many more ...\n',
 'feature_names': ['sepal length (cm)',
  'sepal width (cm)',
  'petal length (cm)',
  'petal width (cm)']
```

```
from IPython.display import Image, display

url = "https://www.googleapis.com/download/storage/v1/b/kaggle-user-content/o/inbox%2F19517213%2F157356294f32ae95956a495960b7ea
display(Image(url=url))
```

```
# Convertendo em Dataframe
df = pd.DataFrame(iris.data, columns = iris.feature_names)
df["target"] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Próximas etapas: Gerar código com df New interactive sheet

```
# contagem das espécies
df.target.value_counts()
```

target	count
0	50
1	50
2	50

dtype: int64

Preparação do dataset para treinar o modelo KNN

```
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Treinamento do modelo

```
modelo = KNeighborsClassifier()
modelo.fit(X_train, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier()
```

Predições

```
pred = modelo.predict(X_test)
print('pred  :', pred[:10])
print('y_test:', y_test[:10])
```

```
pred  : [1 0 2 1 1 0 1 2 1 1]
y_test: [1 0 2 1 1 0 1 2 1 1]
```

```
from sklearn.metrics import accuracy_score
import pandas as pd

# ==========================================
# 1. Testando o modelo para k = 1
# ==========================================
modelo_k1 = KNeighborsClassifier(n_neighbors=1)
modelo_k1.fit(X_train, y_train)
pred_k1 = modelo_k1.predict(X_test)
acuracia_k1 = accuracy_score(y_test, pred_k1)

# ==========================================
# 2. Testando o modelo para k = 3
# ==========================================
modelo_k3 = KNeighborsClassifier(n_neighbors=3)
modelo_k3.fit(X_train, y_train)
pred_k3 = modelo_k3.predict(X_test)
acuracia_k3 = accuracy_score(y_test, pred_k3)

# ==========================================
# 3. Testando o modelo para k = 5
# ==========================================
modelo_k5 = KNeighborsClassifier(n_neighbors=5)
modelo_k5.fit(X_train, y_train)
pred_k5 = modelo_k5.predict(X_test)
acuracia_k5 = accuracy_score(y_test, pred_k5)

# ==========================================
# 4. Testando o modelo para k = 7
# ==========================================
modelo_k7 = KNeighborsClassifier(n_neighbors=7)
modelo_k7.fit(X_train, y_train)
pred_k7 = modelo_k7.predict(X_test)
acuracia_k7 = accuracy_score(y_test, pred_k7)

# ==========================================
# 5. Testando o modelo para k = 9
# ==========================================
modelo_k9 = KNeighborsClassifier(n_neighbors=9)
modelo_k9.fit(X_train, y_train)
pred_k9 = modelo_k9.predict(X_test)
acuracia_k9 = accuracy_score(y_test, pred_k9)

# ==========================================
# Criando a Tabela Comparativa
# ==========================================
# Usamos parênteses () no lugar de colchetes [] para agrupar os dados sem usar listas
dados_para_tabela = {
    'Valor de k': (1, 3, 5, 7, 9),
    'Acurácia': (acuracia_k1, acuracia_k3, acuracia_k5, acuracia_k7, acuracia_k9)
}
```

```
tabela_comparativa = pd.DataFrame(dados_para_tabela)

# Exibe a tabela na tela
tabela_comparativa
```

	Valor de k	Acurácia
0	1	1.000000
1	3	1.000000
2	5	1.000000
3	7	0.966667
4	9	1.000000

Próximas etapas: Gerar código com `tabela_comparativa` New interactive sheet